Rising Waters: AI-Powered Flood Prediction System

Date	04 July 2026
Team ID	
Project Name	Rising Waters – Flood Prediction System
Maximum Marks	Documentation

Project Description

Rising Waters is a Machine Learning-based web application that predicts flood risk from rainfall and weather data. It includes a Data Preprocessing & Training module that builds a Random Forest Classifier from historical rainfall data, and a Flask-based Prediction module that serves real-time, user-friendly flood-risk results through a simple web form.

Using scikit-learn's RandomForestClassifier trained on a Kaggle rainfall dataset, Rising Waters processes ten weather-related inputs (temperature, humidity, cloud cover, and seasonal/annual rainfall figures) to instantly classify a location's flood risk. Built with Flask and a lightweight HTML/CSS frontend, the platform is designed to be simple, fast, and easy to extend with real-time weather feeds in the future.

Scenarios

Scenario 1: A resident notices unusually heavy rainfall during the monsoon and enters the current temperature, humidity, cloud cover, and seasonal rainfall figures into the web form. The system instantly returns a "Flood Likely" result, prompting the resident to take precautionary action.

Scenario 2: A local disaster-management officer wants a quick second opinion before issuing an advisory. They input the latest weather-station readings and receive a "No Flood" prediction, supporting a decision to hold off on an evacuation notice.

Scenario 3: A student researching disaster-management tools tests the application with historical extreme-rainfall values from a past flood event and confirms that the model correctly flags it as high risk, validating the model's usefulness as a teaching example.

Technical Architecture

Rising Waters uses a simple three-layer architecture: an HTML/CSS presentation layer for data entry and result display, a Flask backend that validates input and orchestrates prediction, and a Machine Learning inference layer built on a trained Random Forest model loaded via Joblib. Historical rainfall data (data/flood_dataset.xlsx) is used offline to train the model, which is then serialized to models/flood_model.pkl for fast loading at request time.

Pre-requisites

- Python Programming Proficiency
- Flask Framework Knowledge
- Basic Machine Learning / scikit-learn familiarity
- HTML & CSS skills
- Version Control with Git
- Jupyter Notebook / VS Code development environment setup

Project Workflow

Milestone 1: Data Collection & Preprocessing

- Activity 1.1: Source the rainfall dataset from Kaggle (arbethi/rainfall-dataset).
- Activity 1.2: Load the dataset with Pandas and inspect features: Temperature, Humidity, Cloud Cover, Annual/seasonal Rainfall figures, and the Flood target label.
- Activity 1.3: Split the data into training and test sets using train_test_split (80/20 split, random_state=42).

Milestone 2: Model Development

- Activity 2.1: Train a RandomForestClassifier on the training data (train_model.py).
- Activity 2.2: Evaluate the model on the held-out test set.
- Activity 2.3: Serialize the trained model to models/flood_model.pkl using Joblib for fast reuse.

Milestone 3: Backend Development (app.py & predict.py)

- Activity 3.1: Build predict.py to load the trained model and expose a predict_flood(data) function that accepts a 10-value feature array.
- Activity 3.2: Build app.py with two Flask routes: "/" renders the input form, and "/predict" (POST) reads form values, converts them to floats, calls predict_flood(), and renders the result.
- Activity 3.3: Map the numeric prediction (0 or 1) to a user-friendly label: "✅ No Flood" or "⚠️ Flood Likely".

Milestone 4: Frontend Development

- Activity 4.1: Design templates/index.html with a form for the 10 required weather inputs (Temp, Humidity, Cloud Cover, ANNUAL, Jan-Feb, Mar-May, Jun-Sep, Oct-Dec, avgjune, sub).
- Activity 4.2: Style the page with static/style.css for a clean, readable layout.
- Activity 4.3: Display the prediction result on the same page once the form is submitted (using Flask's render_template with a prediction variable).

Milestone 5: Testing & Local Deployment

- Activity 5.1: Install dependencies from requirements.txt inside a virtual environment.
- Activity 5.2: Run python train_model.py to generate the trained model file.
- Activity 5.3: Run python app.py and test the app locally at http://127.0.0.1:5000 with both normal and heavy-rainfall input values.

Conclusion

Rising Waters demonstrates how a Random Forest Classifier can be packaged into a simple, fast Flask web application to deliver real-time flood-risk predictions from rainfall and weather data. The project covers the full pipeline from data preprocessing and model training to a working web interface, and lays a solid foundation for future enhancements such as live weather API integration, GIS-based flood mapping, and cloud deployment.